Notes on Quotient Inductive Inductive Types

Apiros3

Contents

1 Introduction **2**

2 Quotient Inductive Inductive Types **2**
2.1 Type Theory of Signatures . . . 2
2.1.1 Substitution Calculus . . . 2
2.2 Algebras . . . 3

3 MLTT **3**
3.1 In Functional Programming . . . 3
3.2 As an Inductive-Inductive Type . . . 3
3.2.1 Nat as an Algebra . . . 3
3.3 Metatheory to Algebra . . . 4

4 Other Prerequisites **5**
4.1 MLTT . . . 5
4.1.1 Dependent Types . . . 5
4.1.2 Universes . . . 6
4.2 Terms, Types, and Prop . . . 8
4.3 First Order Logic to Type Theory . . . 8
4.4 Intrinsic and Extrinsic Representation . . . 8
4.5 Interpretation . . . 8
4.5.1 Set Interpretation . . . 8
4.5.2 Display Algebra / Dependent Algebra . . . 10
4.5.3 Morphism . . . 11
4.5.4 Modified Dependent Interpretation . . . 11
4.5.5 Fiber? . . . 11
4.6 Examples . . . 11
4.6.1 Nat (Single-sorted, No-quotient) . . . 11
4.6.2 Contexts and Type (Inductive-Inductive, No Quotient) . . . 12
4.7 Interpretation, Categorically . . . 12
4.7.1 Signatures / Endofunctors . . . 12
4.8 Equivalences, Choice . . . 13

5 Todo **13**

6 Other **14**
6.1 Curry-Howard Correspondence . . . 14
6.2 LEM? . . . 14
6.2.1 Computability . . . 14
6.2.2 Inderivability . . . 14

1 Introduction

Natural numbers act quite often as a very useful tool as a first read on new concepts. Here I attempt to write a list of concepts I've encountered, and how $\mathbb{N}$ acts as a useful tool in giving a concrete example of what is happening, before the idea is further abstracted.

Stuff here is mainly based on the QIIT paper by Ambrus Kaposi.

2 Quotient Inductive Inductive Types

2.1 Type Theory of Signatures

We consider a QIIT with four sorts, $\mathsf{Ty}\ \Gamma$ denotes the well-formed types with free variables in Γ. An element of $\mathsf{Sub}\ \Gamma\ \Delta$ is a list of terms where it contains one term for each type in Δ and each of these have free variables in Γ. An element of $\mathsf{Tm}\ \Gamma\ A$ is a term of type A with free variables in Γ

Con	:	Set	contexts
Ty	:	$\mathsf{Con} \to \mathsf{Set}$	types
Sub	:	$\mathsf{Con} \to \mathsf{Con} \to \mathsf{Set}$	substitutions
Tm	:	$(\Gamma : \mathsf{Con}) \to \mathsf{Ty}\,\Gamma \to \mathsf{Set}$	terms

2.1.1 Substitution Calculus

The following constructors give the substitution calculus part of the syntax.

$\cdot$	:	Con	empty context
$- \triangleright -$	:	$(\Gamma : \mathsf{Con}) \to \mathsf{Ty}\,\Gamma \to \mathsf{Con}$	context extension
$-[-]$	:	$\mathsf{Ty}\,\Delta \to \mathsf{Sub}\,\Gamma\,\Delta \to \mathsf{Ty}\,\Gamma$	substitution of types
id	:	$\mathsf{Sub}\,\Gamma\,\Gamma$	identity substitution
$- \circ -$	:	$\mathsf{Sub}\,\Theta\,\Delta \to \mathsf{Sub}\,\Gamma\,\Theta \to \mathsf{Sub}\,\Gamma\,\Delta$	composition
ε	:	$\mathsf{Sub}\,\Gamma\,\cdot$	empty substitution
$-, -$	:	$(\sigma : \mathsf{Sub}\,\Gamma\,\Delta) \to \mathsf{Tm}\,\Gamma\,A[\sigma] \to \mathsf{Sub}\,\Gamma\,(\Delta \triangleright A)$	substitution extension
π_1	:	$\mathsf{Sub}\,\Gamma\,(\Delta \triangleright A) \to \mathsf{Sub}\,\Gamma\,\Delta$	first projection
π_2	:	$(\sigma : \mathsf{Sub}\,\Gamma\,(\Delta \triangleright A)) \to \mathsf{Tm}\,\Gamma\,A[\pi_1\sigma]$	second projection
$-[-]$	:	$\mathsf{Tm}\,\Delta\,A \to (\sigma : \mathsf{Sub}\,\Gamma\,\Delta) \to \mathsf{Tm}\,\Gamma\,A[\sigma]$	substitution of terms
[id]	:	$A[\mathrm{id}] = A$	
$[\circ]$	:	$A[\sigma \circ \delta] = A[\sigma][\delta]$	
ass	:	$(\sigma \circ \delta) \circ \nu = \sigma \circ (\delta \circ \nu)$	
idl	:	$\mathrm{id} \circ \sigma = \sigma$	
idr	:	$\sigma \circ \mathrm{id} = \sigma$	
$\cdot\eta$	:	$\{\sigma : \mathsf{Sub}\,\Gamma\,\cdot\} \to \sigma = \varepsilon$	
$\triangleright\beta_1$	:	$\pi_1(\sigma, t) = \sigma$	
$\triangleright\beta_2$	:	$\pi_2(\sigma, t) = t$	
$\triangleright\eta$	:	$(\pi_1\ \sigma, \pi_2\ \sigma) = \sigma$	
$,\circ$	:	$(\sigma, t) \circ \delta = (\sigma \circ \delta,\ t[\delta])$	

The main syntax is captioned, while the laws are given explicit names and types. The operations are what one would expect, including the β-reduction and η-expansion rules.

2.2 Algebras

We give the notion of algebras by induction on the syntax of the theory of Signatures. We have:

$$\begin{aligned}(\Gamma : \mathsf{Con})^A &: \mathsf{Set}\\ (A : \mathsf{Ty}\,\Gamma)^A &: \Gamma^A \to \mathsf{Set}\\ (\sigma : \mathsf{Sub}\,\Gamma\,\Delta)^A &: \Gamma^A \to \Delta^A\\ (t : \mathsf{Tm}\,\Gamma\,A)^A &: (\gamma : \Gamma^A) \to A^A\,\gamma\end{aligned}$$

3 MLTT

3.1 In Functional Programming

In Haskell, creating an object that behaves in a way that we understand the natural numbers is straightforward. Simply,

```
data Nat where
    Zero  :: Nat
  | Succ  :: Nat → Nat
```

or isomorphically,

```
data Nat where
    Zero  :: ()  → Nat
  | Succ  :: Nat → Nat
```

3.2 As an Inductive-Inductive Type

Metatheoretically, in the context of Type Theory, we have

```
Nat : Type
zero : Nat
succ : Nat → Nat
```

Definition 3.2.1. *An **Inductive-Inductive Type** simultaneously introduces a type* `A : Type` *and an* `A`*-indexed family* `B : A → Type` *both by induction, where constructors of* `A` *can mention* `B` *and constructors of* `B` *can mention* `A` *in strictly positive ways.*

3.2.1 Nat as an Algebra

The following three element context is the signature for natural numbers. It has one sort and two operators:

$$\Delta :\equiv (Nat : \mathtt{U}, zero : \mathtt{El}\ Nat, suc : Nat \Rightarrow \mathtt{El}\ Nat)$$

Writing $-^A$ for the set of algebras given a signature, we can compute

$$\Delta^{\mathrm{A}} \equiv (N : \mathtt{Set}) \times N \times (N \to N)$$

We write the initial algebra by $\mathrm{con}_\Delta : \Delta^{\mathrm{A}}$. Explicitly, we have

$$\mathrm{con}_\Delta \equiv (\mathtt{Tm}\ \Delta\ (\mathtt{El}\ Nat),\ zero,\ \lambda t.suc\ \mathtt{@}\ t)$$

The dependent algebra over an algebra (N, z, s), written $-^{\mathrm{D}}$, consists of the proof-relevant predicate over N, a witness of the predicate at z and a proof that s respects the predicate. Thus,

$$\Delta^{\mathrm{D}}(N, z, s) \equiv (N^{\mathrm{D}} : N \to \mathtt{Set} \times N^{\mathrm{D}}\ z \times ((x : N)\ \to N^{\mathrm{D}}\ x \to N^{\mathrm{D}}\ (s\ x)))$$

3.3 Metatheory to Algebra

A signature on `Nat` looks something like

```
Nat  : Set
zero : Nat
succ : Nat → Nat
```

Then we can define algebras for this signature:

```
Record Nat_A : Type := {
    N : Set;
    z : A;
    s : A → A
}
```

We can realize this signature as an inductive type in the usual way, which generates the initial algebra:

```
Inductive nat : Set :=
    | O : nat
    | S : nat → nat.

Definition I : Nat_A := {|
    N := nat;
    z := O;
    s := S
|}
```

The displayed `Nat`-algebra over I is a family over the carrier set together with display evidence matching the operators. Thus, we can write

```
Record Nat_D (Γ : Nat_A) : Type := {
    N_D : N Γ → Set;
    z_D : N_D (z Γ);
    s_D : forall n, N_D n → N_D (s Γ n)
}.
```

Then, we can give the induction principle:

```
Fixpoint indNat {Γ : Nat_A} (Γ_D : Nat_D Γ) (n : N Γ) : N_D Γ_D n :=
    match n with
        | O    ⇒ z_D Γ_D
        | S n' ⇒ s_D Γ_D n' (indNat Γ_D n')
```

Omitting the explicit algebra for readibility, up to abuse of notation, we have

```
Variable Γ : Nat_A.
Variable Γ_D : Nat_D Γ.

Record Nat_D : Type := {
    N_D : N → Set;
    z_D : N_D z;
    s_D : forall n, N_D n → N_D (s n)
}.

Fixpoint indNat (n : N) : N_D n :=
    match n with
        | O    ⇒ z_D
        | S n' ⇒ s_D n' (indNat n')
```

which gives us the standard induction on `Nat` for an arbitrary `Γ : Nat_A`, given that we can provide an instance for the corresponding `Nat_D Γ`.

For example, consider

4 Other Prerequisites

4.1 MLTT

4.1.1 Dependent Types

Definition 4.1.1. *A **dependent type** is a type that can mention / depend on a term.*

If $A : \texttt{Type}$ and we have a type family $B : A \to \texttt{Type}$, then for every $a : A$, we have a fiber $B(a)$. Generally, we have dependent Π-types (dependent functions) and Σ-types (dependent pairs).

A Π-type with notation $\Pi_{x:A}B(x)$ is the type of functions that given a $x : A$, returns a result in the corresponding fiber $B(x)$. If B doesn't depend on x, then $\Pi_{x:A}B \simeq A \to B$. This comes along with formation,

$$\frac{\Gamma \vdash A : \texttt{Type} \qquad \Gamma, x : A \vdash B : \texttt{Type}}{\Gamma \vdash \Pi(x : A).B : \texttt{Type}}$$

Introduction (pairing),

$$\frac{\Gamma, x : A \vdash b : B}{\Gamma \vdash \lambda x.b : \Pi(x : A).B}$$

Elimination (application),

$$\frac{\Gamma \vdash f : \Pi(x : A).B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[a/x]}$$

Computation (β),

$$(\lambda x.b)a \equiv b[a/x]$$

Often alongside the η-principle (uniqueness) at a judgement or propositional level, though not included in vanilla MLTT.

Under the Curry-Howard correspondence, $\Pi_{x:A}B(x)$ corresponds to $\forall x : A.B(x)$. That is, a term of this type is a function that given a witness $x : A$, produces a proof of $B(x)$.

A Σ-type with notation $\Sigma_{x:A}B(x)$ is the type of pairs where the second component's type depends on the first. An element is (a, b) with $a : A$ and $b : B(a)$. If B does not depend on x, then $\Sigma_{x:A}B \simeq A \times B$. Similar to Π-types, this comes with formation,

$$\frac{\Gamma \vdash A : \texttt{Type} \qquad \Gamma, x : A \vdash B : \texttt{Type}}{\Gamma \vdash \Sigma(x : A).B : \texttt{Type}}$$

Introduction (pairing),

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash b : B[a/x]}{\Gamma \vdash (a, b) : \Sigma(x : A).B}$$

Elimination (projection / dependent eliminator),

$$\texttt{fst} : (\Sigma(x : A).B) \to A \qquad \texttt{snd} : \prod_{p:\Sigma(x:A).B} B(\texttt{fst}\ p)$$

In particular, for $p : (\Sigma(x : A).B)$, we have $\texttt{fst}\ p : A$ and $\texttt{snd}\ p : B(\texttt{fst}\ p)$. Then we have computation (β),

$$\texttt{fst}(a, b) \equiv a \qquad \texttt{snd}(a, b) \equiv b$$

Under the curry-howard isomorphism, $\Sigma_{x:A}B(x)$ corresponds to $\exists x : A.B(x)$ constructively. That is, a witness a together with a proof $B(a)$.

In the context of sets, given a family $B : A \to \texttt{Set}$, we get

- $\Pi_{x\in A}B_x$: choice functions picking $b_x \in B_x$ for each $x \in A$
- $\Sigma_{x\in A}B_x$: disjoint union $\biguplus_{x\in A}\{x\} \times B_x$

Categorically, for a map $f : X \to A$, pulling back families along f has adjoints:

- Right adjoint Π_f : dependent product (universal quantification)
- Left adjoint Σ_f : dependent sum (existential / pushforward)

Some intuitive examples:

- $\texttt{Vec}(A, n) : \texttt{Type}$ depends on $n : \mathbb{N}$
  - Head has type $\Pi_{n:\mathbb{N}}\Pi_{v:\texttt{Vec}(A,\texttt{suc}\ n)}A$
  - A value with its length lives in $\Sigma_{n:\mathbb{N}}\texttt{Vec}(A, n)$
- A sorted list with a proof it is sorted would have the type $\Sigma_{\ell:\texttt{List}\ A}\ \texttt{Sorted}(\ell)$

4.1.2 Universes

We use the notion of *types of types* to avoid size paradoxes (i.e. Russell's paradox). Hence we use a predicative ladder

$$\texttt{Type}_0 : \texttt{Type}_1 : \texttt{Type}_2 : \cdots$$

usually along with a notion of cumulativity, such that if $A : \texttt{Type}_i$ then $A : \texttt{Type}_{i+1}$. Then by universe polymorphism, we can write level-generic code.

We have (generally) two universe styles, the Russell-style and Tarski-style. Over a Russell-style universe, judgements are of the form $\texttt{A} : \texttt{Type}_i$, such that type formers return types directly:

$$\frac{A : \texttt{Type}_i \qquad (x : A) \vdash B : \texttt{Type}_j}{\prod(x : A).B : \texttt{Type}_{\max(i,j)}}$$

In that sense, the universe acts like a set of types, and it's members are types. Hence the saying "types are elements".

Contrary to this, over a Tarski-style universe, the universe is instead a *type of codes* $\texttt{U}_i : \texttt{Type}_{i+1}$ plus some decoding $\texttt{El}_i : \texttt{U}_i : \texttt{Type}_i$. Closure under type formers is then given by *codes*. We make this formal:

Definition 4.1.2. *A **Tarski-Style Universe** is a type* `U : Type` *whose elements are codes (names) for small types.*

Definition 4.1.3. *An **element**, written* `El` *is a function that turns a code into the type it names. Explicitly, we have* `El : U → Type` *such that if* `u : U` *is a code, then* `El u` *is the actual type it denotes.*

In a Tarski Universe, we inductively define what types of codes exist, and recursively say what that means as an element of `Type` via `El` (so to be more precise, a Tarski Style Universe is a pair `(U, El)`). The `El` ensures that the constructors of the datatype are strictly positive. For example, we can consider the following toy example:

```
data U : Type where
    uUnit  : U
    uSigma : (a : U) → (El a → U) → U
    uPi    : (a : U) → (El a → U) → U
```

Defines a small universe explicitly, then we define

```
El : U → Type
El uUnit        = Unit
El (uSigma a b) = Σ (x : El a), El (b x)
El (uPi a b)    = (x : El a) → El (b x)
```

which gives concrete meaning to `uSigma` and `uPi`. We write `El u1 = Unit` for simplicity.

Tarski Style Universes aid in internalizing a general schema for inductive-inductive types, as this universe encode allowed constructors, indices, parameters, etc, and from which one can interpret the actual data types as initial algebras.

Over a Russell Style Universe, types are elements of $\texttt{Type}_\ell$, but there is no data that can be inspected. In plain MLTT, we can define a function like

$$F : \texttt{Type}_\ell \to \texttt{Type}_\ell \qquad F\ A = A \times A$$

but cannot define a function

$$G : \texttt{Type}_\ell \to \mathbb{N}$$

by "pattern matching" on A. That is, there is no eliminator that reveals the structure of an arbitrary $A : \texttt{Type}$. In contrast, over a Tarski style universe, as `U` is an inductive family of codes, we can do recursive pattern matching / induction.

As a toy model, we can a Russell style identity function which probably looks like:

```
id : Π (A : Type U), A → A
id A x = x
```

whereas under a Tarski style it would look like

```
id : Π (A : U), El A → El A
id A x = x
```

4.2 Terms, Types, and Prop

4.3 First Order Logic to Type Theory

4.4 Intrinsic and Extrinsic Representation

In general, an extrinsic representation is one which has some independent separation between the object and a property/proof. In contrast, an intrinsic representation will carry that property inside that definition. For instance, the code

```
xs : List A
p : length xs = 5
```

using `List` makes the property that `xs` has length 5 becomes extrinsic. In contrast, if we just had some

```
xs : Vec A 5
```

Then the length of the list is an intrinsic property that is embedded within the construction itself.

Lambda calculus has an extrinsic representation where terms are made by an abstract syntax tree, and then we give an external typing judgement to rule out unwanted terms (like Ω).

We give a toy example of this. First consider the raw syntax of types, contexts, variables, and terms:

```
A ::=  A ⇒ A | Bool
Γ ::= ◇ | Γ, A
x ::= ∅ | suc x
t ::= x | lam x | t @ t | true | false | ite t t t
```

This clearly contains unwanted terms like `true @ false`, which we remove by a typing judgement, in our case the following:

In contrast we can make this intrinsic by defining terms to be dependent on the context and type like

```
data Var : Con → Ty → Set where
    ∅   : Var (Γ, A) A
    suc : Var Γ A → Var (Γ, B) A

data Tm : Con → Ty → Set where
    var : Var Γ A → Tm Γ A
    lam : Tm (Γ, A) B → Tm Γ (A → B)
    _@_ : Tm Γ (A → B) → Tm Γ A → Tm Γ B
```

If we want β-reduction, we can just quotient by the equality

```
β : lam t @ u = t [id, u]
```

4.5 Interpretation

4.5.1 Set Interpretation

Definition 4.5.1. *The **set interpretation** is the interpretation into the set model. The operation "$[\![-]\!]$" gives the set interpretation, given a signature and a context/term.*

The set interpretation of types is what one would expect, where $[\![A]\!] : \texttt{Set}$ and we have the standard rules like

$$\begin{aligned}
[\![A \Rightarrow B]\!] &= [\![A]\!] \to [\![B]\!] \\
[\![\texttt{Prod}\ A\ B]\!] &= [\![A]\!] \times [\![B]\!] \\
[\![\texttt{Bool}]\!] &= \mathbb{2} \\
[\![\texttt{Nat}]\!] &= \mathbb{N} \\
[\![\texttt{Unit}]\!] &= \mathbb{1}
\end{aligned}$$

Then, the set interpretation of contexts is also straightforward, recursively defined as

$$[\![\Gamma]\!] : \texttt{Set} \qquad \begin{cases} [\![\diamond]\!] = \mathbb{1} \\ [\![\Gamma, A]\!] = [\![\Gamma]\!] \times [\![A]\!] \end{cases}$$

Finally, the set interpretation of terms depends on the context and returns the type based on it, so when we have $\Gamma \vdash t : A$, we should have

$$[\![t]\!] : [\![\Gamma]\!] \to [\![A]\!]$$

As a sanity check, consider $\Gamma, A \vdash \emptyset : A$. We get what we expect,

$$[\![\emptyset]\!] : [\![\Gamma]\!] \times [\![A]\!] \to [\![A]\!] \qquad [\![\emptyset]\!] = \text{snd}$$

Now recall the inference rule

$$\frac{\Gamma \vdash x : A}{\Gamma, B \vdash \texttt{suc}\ x : A}$$

So we have

$$[\![\texttt{suc}\ x]\!] : [\![\Gamma]\!] \times [\![B]\!] \to [\![A]\!]$$

given

$$[\![x]\!] : [\![\Gamma]\!] \to [\![A]\!]$$

Thus the only reasonable function we have is

$$[\![\texttt{suc}\ x]\!] = [\![x]\!] \circ \text{fst}$$

As another example, consider the abstraction rule

$$\frac{\Gamma, A \vdash t : B}{\Gamma \vdash \texttt{lam}\ t : A \Rightarrow B}$$

Then we have that

$$[\![\texttt{lam}\ t]\!] : [\![\Gamma]\!] \to [\![A \Rightarrow B]\!] = [\![\Gamma]\!] \to [\![A]\!] \to [\![B]\!]$$

along with

$$[\![t]\!] : [\![\Gamma]\!] \times [\![A]\!] \to [\![B]\!]$$

Hence

$$[\![\texttt{lam}\ t]\!] = \text{curry}_2\ [\![t]\!]$$

where the curry_2 function is one which currys a function with a 2-tuple input. More intuitively,

$$[\![\texttt{lam}\ t]\!]\gamma = \lambda\alpha.[\![t]\!](\gamma, \alpha)$$

In a dependent type theory setting, types can depend on the context. Hence, while $[\![\Gamma]\!] : \mathtt{Set}$, we expect $[\![A]\!] : [\![\Gamma]\!] \to \mathtt{Set}$. Terms hence depend on a context and returns a type that depends on the input context. Explicitly, we have $[\![t]\!] : (\gamma : [\![\Gamma]\!]) \to [\![A]\!]\gamma$.

Consider the usual example

$$\mathtt{Vec}_{\mathtt{Bool}} : \mathtt{Nat} \to \mathtt{Type}$$

alongisde a function `trues` that given a natural number returns a vector of that length with only trues:

$$\mathtt{trues} : (n : \mathtt{Nat}) \to \mathtt{Vec}_{\mathtt{Bool}}\ n$$

We can then type something like

$$\diamond, \mathtt{Nat} \vdash \mathtt{trues}\ \mathtt{@}\ \emptyset : \mathtt{Vec}_{\mathtt{Bool}}\emptyset$$

Then we have

$$[\![\mathtt{trues}\ \mathtt{@}\ \emptyset]\!] : (\gamma : [\![\diamond, \mathtt{Nat}]\!]) \to [\![\mathtt{Vec}_{\mathtt{Bool}}\emptyset]\!]\gamma$$

Noting that $[\![\diamond, \mathtt{Nat}]\!] = \top \times \mathbb{N}$ and $[\![\mathtt{Vec}_{\mathtt{Bool}}\emptyset]\!]\gamma = \mathbb{V}\mathrm{ec}_2([\![\emptyset]\!]\ \gamma)$, this simplifies to

$$[\![\mathtt{trues}\ \mathtt{@}\ \emptyset]\!] : (\gamma : \top \times \mathbb{N}) \to \mathbb{V}\mathrm{ec}_2([\![\emptyset]\!]\ \gamma)$$

Rewriting $\gamma := (tt, n)$ and noting that $[\![\emptyset]\!] = \mathrm{snd}$, we have

$$[\![\mathtt{trues}\ \mathtt{@}\ \emptyset]\!] : ((tt, n) : \top \times \mathbb{N}) \to \mathbb{V}\mathrm{ec}_2\ n$$

4.5.2 Display Algebra / Dependent Algebra

Given an algebra, the display algebra is the family version of it that lives over it, such that it can model dependent elimination.

Here, a context is interpreted as a predicate $\Gamma^{\mathrm{D}} : \Gamma^{\mathrm{A}} \to \mathtt{Set}$. A type $A : \mathtt{Ty}\ \Gamma$ becomes a predicate that depends on a witness of Γ^{D}, which explictly is $A^{\mathrm{D}}\ \{\gamma : \Gamma^{\mathrm{A}}\} : \Gamma^{\mathrm{D}}\ \gamma \to A^{\mathrm{A}}\ \gamma \to \mathtt{Set}$. Finally, a term $t : \mathtt{Tm}\ \Gamma\ A$ is interpreted as $t^{\mathrm{D}}\ \{\gamma : \Gamma^{\mathrm{A}}\} : (\gamma^{\mathrm{D}} : \Gamma^{\mathrm{D}}\ \gamma) \to A^{\mathrm{D}}\ \gamma^{\mathrm{D}}\ (t^{\mathrm{A}}\ \gamma)$. Intuitively, t^{A} says what the value of t is in the base algebra, and t^{D} says how to lift t into the display algebra (that is, given a display evidence γ^{D} for the context, it produces a display evidence that t has type A, or equivalently a witness in the fiber $A^{\mathrm{D}}\ \gamma^{\mathrm{D}}\ (t^{\mathrm{A}}\ \gamma)$).

For instance, given as an algebraic signature (metatheory), we have

```
Bool : Set
true : Bool
false : Bool
```

An algebra for this signature can be seen as

```
Record Bool_alg : Type := {
    A : Set;
    true_alg : A;
    false_alg : A
}.
```

We can think of the inductive `Bool` type as

```
Inductive Bool_I : Set
    | true_I : Bool_I
    | false_I : Bool_I.
```

which we can then package as

```
Definition I : Bool_alg := {|
    A := Bool;
    true_alg := true;
    false_alg := false
|}.
```

Then this induces a displayed algebra

```
Bool• : Bool_I → Set
true• : Bool• true_I
false• : Bool• false_I
```

which gives the induction principle on bool:

```
ind_I : (b : Bool_I) → Bool• b
```

4.5.3 Morphism

An algebra homomorphism is given by a function between sets which respects operators, with propositional equality being denoted $=$.

We write $-^{\mathrm{M}}$ to represent morphisms. If Δ is an algebra, we have $\Delta^{\mathrm{M}} : \Delta^{\mathrm{A}} \to \Delta^{\mathrm{A}} \to \texttt{Set}$. We interpret contexts as a relation $\Gamma^{\mathrm{M}} : \Gamma^{\mathrm{A}} \to \Gamma^{\mathrm{A}} \to \texttt{Set}$.

4.5.4 Modified Dependent Interpretation

4.5.5 Fiber?

Definition 4.5.2. *A **fiber** of an element y under a function f is the preimage of the singleton set $\{y\}$.*

In the scope of these notes, it is the set that sits over a base element in the family. Explicitly, if $B : A \to \texttt{Set}$ is a family, then for each $a : A$, the fiber of B over a is just $B(a)$.

In the context of this subsection, given an inductive type W, the base algebra gives an element $w : W$, given a dependent family $P : W \to \texttt{Type}$ with constructor cases (inductive hypotheses), the display algebra gives the fiber $P(w)$, and the display evidence is an element of that fiber. For instance, for $P : \mathbb{N} \to \texttt{Set}$, the fiber over 3 is $P(3)$, and a proof $p : P(3)$ is the display evidence living in that fiber.

4.6 Examples

4.6.1 Nat (Single-sorted, No-quotient)

We consider how the algebras interact for Nat. The algebraic signature (metatheory) for Nat is given by, one sort and two point-constructors:

```
data Nat : Set where
    zero :: Nat
  | succ :: Nat → Nat
```

The algebra (set interpretation) is what one would expect with the familiar triple, given by:

- The carrier `N : Set`
- `z : N`

- `s : N → N`

Homomorphisms between Nat, are given in the usual sense as well. That is, between `(N, z, s)` and `(N', z', s')`, we have a map `h : N → N'` such that `h z = z'` and `h (s n) = s' (h n)` for any `n`, where these equalities are propositional in the UIP metatheory.

The displayed algebra over a base point `(N, z, s)` is then

- The family `N• : N → Set` (the motive)
- `z• : N• z`
- `s• : ∀ n. N• n → N• (s n)` (the methods)

The sections contains the dependent primitive recursion:

- `rec• : ∀n. N• n`
- with β-laws `rec• z = z•` and `rec• (s n) = s• n (rec• n)`

From a CwF view:

1. Objects/contexts are the algebras: $\Gamma \equiv$ `(N, z ,s)`
2. Morphisms/substitutions is the homomorphisms between algebras
3. Types over Γ are the displayed algebras `(N•, z•, s•)` over Γ
4. Terms $\Gamma \vdash A$ are sections `rec• : ∀ n. N• n`

4.6.2 Contexts and Type (Inductive-Inductive, No Quotient)

4.7 Interpretation, Categorically

4.7.1 Signatures / Endofunctors

A signature Σ for an inductive object induces an endofunctor $F : \mathcal{C} \to \mathcal{C}$.

We outline some simple examples:

- Naturals

Naturals come with shapes $S = 1 + 1$ corresponding to zero and succ. The positions are given by $P(\mathsf{zero}) = \emptyset$ and $P(\mathsf{succ}) = 1$, which gives the functor $F(X) = 1 + X$, alongisde an induced initial algebra `Nat` with `zero : Nat` and `suc : Nat -> Nat`. As usual, we can write this out explicitly:

```
data Nat : Set where
    zero :: Nat
  | succ :: Nat → Nat
```

- Lists over some A

Given a fixed $A : \mathsf{Type}$, we have shapes $S = 1 + A$ corresponding to nil and $\mathsf{cons}\ a$. Positions are then given by $P(\mathsf{nil}) = \emptyset$ and $P(\mathsf{cons}\ a) = 1$, which gives the functor $F(X) = 1 + A \times X$, giving initial algebra

```
data List (A : Set) : Set where
    nil  :: List A
    cons :: A → List A → List A
```

- Binary Trees with B-laveled leaves

Given a fixed $B : \mathsf{Type}$, we have shapes $S = B + 1$ corresponding to leaf and node (functionally we can view this like $S \simeq B \uplus 1$ and `inl b` to mean leaf with label b and `inr *` to mean the binary node). We have positions $P(\mathsf{inl}\ b) = \emptyset$ and $P(\mathsf{inr}\ *) = 2\!\!1$.

This gives a polynomial functor

$$F(X) = B + X \times X$$

Alongside the initial algebra

```
data Tree (B : Set) : Set where
    leaf : B → Tree B
    node : Tree B → Tree B → Tree B
```

```
record Container : Set1 where
    field Shape : Set
          Pos   : Shape → Set
```

4.8 Equivalences, Choice

TODO: ext, eta, strength, choice?

5 Todo

1. Free monoid over A is just a List
2. Quotienting representation without quotients (eg. integer, lambda calculus)
3. Decidability (some things can't be done, say CL-logic, real numbers, no normal form (else halting))
4. $\Sigma(A : Set)(A \to I) \simeq (I \to Set)$
5. locally cartesian closed categories
6. Dependent algebra can capture equality as well, useful for structural induction (consider the leaf counting function and node counting function)

6 Other

6.1 Curry-Howard Correspondence

6.2 LEM?

The **Law of Excluded Middle (LEM)** is the schema that

$$\forall P. P \vee \neg P$$

Under the Curry-Howard correspondence, this is a type that is inhavited precisely when every proposition P is *decidable*.

6.2.1 Computability

Under Curry-Howard, a proof of Σ `x:A. P x` is a program that returns a concrete `x:A` plus evidence of `P x`. Similarly, a proof of `P ⊎¬ P` is a program that decides `P`. Unrestricted LEM says `∀ P. P ⊎¬ P`. This would be a single, uniform decision procedure for every proposition `P`. Such a procedure cannot exist in a purely computable setting (as it would decide the halting problem, etc.). Hence, mainstream constructive type theories (like MLTT/Agda/Coq's core) do not validate LEM by computation.

When LEM is postulated at the computational level (`Type` / `Set`), we obtain a constant with no reduction rule. Then, we can write closed programs whose result depend on evaluating that constant, so the evaluation gets stuck. Consider:

```
postulate
    EM : (P : Set) → P ⊎ (P → ⊥)

coin : ℕ
coin with EM (0 ≡ 1)
... | inj₁ _ = 0
... | inj₂ _ = 1
```

Then, `coin` is a closed $\mathbb{N}$ but does not reduce to a numeral, as it is stuck on `EM`. Hence, even if normalization of existing terms is preserved, addition of postulates makes new closed terms non-caninical. Then, program extraction (in a weak sense) fails.

All in all, these have to do with proof relevance.

6.2.2 Inderivability

LEM is *not derivable* in constructive type theory and is *not refutable* either. Hence this axiom is an independent design-level choice.